

Konstante Definitionen

```
private static final int ROWS = 1000;
```

- **Sichtbarkeit:** `private` → Zugriff nur innerhalb der Klasse.
 - **Modifizierer:**
 - `static` → gehört zur Klasse, nicht zur Instanz.
 - `final` → unveränderlicher Wert nach Initialisierung.
 - **Datentyp:** `int` → primitiver ganzzahliger Typ.
 - **Zweck:** Legt fest, wie viele Zeilen generiert und geschrieben werden. Wert: 1000.
-

```
private static final int COLUMNS = 1000;
```

- Legt fest, wie viele zufällige Zeichen pro Zeile generiert werden. Wert: 1000.
 - Der resultierende Output hat somit **1000 × 1000 = 1.000.000 Zeichen**.
-

```
private static final String FILE_PATH = "random_chars.txt";
```

- Enthält den relativen Pfad (im aktuellen Arbeitsverzeichnis) der Zieldatei.
 - Der Dateiname ist konstant, `.txt`-Endung wird nicht validiert.
-

```
private static final String LINE_SEPARATOR = System.lineSeparator();
```

- Systemabhängiger Zeilenumbruch (z. B. `\n` auf Unix/Linux/macOS, `\r\n` unter Windows).
 - Liefert konsistente Plattformkompatibilität bei Textdateien.
-

```
private static final String CHARACTERS = "dhbwmos";
```

- Alphabet für die Zufallsgenerierung. Zeichenkette besteht aus 7 Buchstaben (`d`, `h`, `b`, `w`, `m`, `o`, `s`).
 - Indexpositionen: `0` bis `6`.
-

```
private static final Random RANDOM_GENERATOR = new Random();
```

- Einmalig instanzierter Zufallszahlengenerator vom Typ `java.util.Random`.
 - Standardmäßig initialisiert mit einem Pseudo-Zufallsseed (z. B. Systemzeit).
 - Wird im gesamten Programm zur Generierung von Zufallszahlen verwendet.
-
-

Hauptmethode (`main`)

@SneakyThrows

- **Lombok-Annotation**, die checked Exceptions unterdrückt.
- In diesem Fall: `IOException`, welche von `BufferedWriter` oder `FileWriter` ausgelöst werden könnte.
- Wird zur Compile-Zeit durch einen `try/catch`-Block ersetzt, der die Exception in eine unchecked Form weiterwirft.

```
public static void main(String... args) {
```

- Einstiegspunkt des Programms. Wird von der JVM aufgerufen.
- `String... args` ist eine Varargs-Notation für Kommandozeilenargumente.
- `public static` → Standardzugriffskonvention für `main`.

```
    try (BufferedWriter writer = new BufferedWriter(new  
        FileWriter(Path.of(FILE_PATH).toFile())) {
```

- **Try-with-resources-Block:** Automatisches Schließen des `BufferedWriter`, selbst bei Exceptions.
- `Path.of(FILE_PATH)` → erzeugt ein `java.nio.file.Path`-Objekt aus dem Dateipfad.
- `.toFile()` → konvertiert `Path` in ein `java.io.File`-Objekt.
- `new FileWriter(...)` → öffnet einen Stream im **Schreibmodus**, ggf. wird die Datei neu angelegt oder überschrieben.
- `new BufferedWriter(...)` → wrappt den `FileWriter`, um das Schreiben durch internen Puffer effizienter zu machen.

```
        IntStream.range(0, ROWS).forEach(i -> writeLine(writer));
```

- Funktionaler Zugriff auf den Indexbereich `0` bis `999` (ROW-Count: 1000).
- Jeder Wert `i` steht für eine Zeile.
- Für jede Iteration wird die Methode `writeLine(writer)` aufgerufen.
- Vorteil: Stream-basierter, deklarativer Schleifenansatz mit Lambda-Ausdruck.

Hilfsmethode `writeLine`

@SneakyThrows

- Gleicht der Beschreibung bei `main`.

- Unterdrückt die `IOException`, die durch `writer.write()` auftreten kann.

```
private static void writeLine(BufferedWriter writer) {
```

- Private statische Hilfsmethode.
- Erwartet einen offenen `BufferedWriter` zur Dateiausgabe.
- Keine Rückgabe (`void`), rein seiteneffektreich.

```
String line = RANDOM_GENERATOR.ints(COLUMNS, 0, CHARACTERS.length())
```

- `ints(n, origin, bound)` erzeugt einen `IntStream` mit `n` Zufallszahlen.
- `COLUMNS` bestimmt die Anzahl der Zeichen.
- Die Zufallszahlen liegen im Bereich `[0, CHARACTERS.length())`, also `0-6`.
- Jeder Wert entspricht einem Index im `CHARACTERS`-String.

```
.mapToObj(CHARACTERS::charAt)
```

- Konvertiert jeden Integer (Index) in das entsprechende Zeichen.
- `charAt(int)` liefert Zeichen an der Position im `CHARACTERS`-String.
- Ergebnis ist ein `Stream<Character>`.

```
.collect(StringBuilder::new, StringBuilder::append, StringBuilder::append)
```

- Reduziert den Stream zu einem einzelnen `StringBuilder`.
- Konstrukturfunktion: `StringBuilder::new`
- Akkumulation: `StringBuilder::append`
- Kombinatorfunktion (parallel irrelevant): ebenfalls `StringBuilder::append`

```
.append(LINE_SEPARATOR)  
.toString();
```

- Hängt systemabhängigen Zeilenumbruch an.
- Konvertiert finalen `StringBuilder` in `String`.

```
writer.write(line);
```

- Schreibt die Zeichenkette (1000 zufällige Buchstaben + Zeilenumbruch) in den Stream.

- Pufferung erfolgt intern über `BufferedWriter`.
-

Zusammenfassung

Das Programm erstellt eine Datei mit 1000 Zeilen, wobei jede Zeile aus 1000 zufälligen Zeichen besteht, gewählt aus dem Alphabet `"dhbwms"`. Der gesamte Text umfasst über 1 Million Zeichen. Die Ausgabe erfolgt effizient gepuffert und mit systemkompatiblen Zeilenumbrüchen. Fehlerbehandlung wird durch Lomboks `@SneakyThrows` automatisiert umgangen.

Prof. Dr. Carsten Müller | Software Engineering